

Problema de los 8 Caballos

1. Descripción del Problema

El problema de los 8 caballos consiste en ubicar exactamente 8 caballos en un tablero de ajedrez de 8×8 casillas de tal forma que ningún caballo pueda atacar a otro. Dado que el caballo se mueve en forma de "L", se deben encontrar 8 posiciones donde ningún par de caballos se encuentre en casillas desde las cuales uno pueda alcanzar al otro con ese movimiento.

El programa debe soportar dos modos de inicio:

- Caso 1 – Tablero vacío: el algoritmo parte desde cero y ubica los 8 caballos.
- Caso 2 – Caballo inicial: el usuario indica una posición de inicio; el algoritmo coloca los 7 caballos restantes respetando esa posición fija.

2. Restricciones del Problema

2.1 Movimiento del caballo

El caballo se mueve dos casillas en una dirección y una en la dirección perpendicular, formando una "L". Los 8 desplazamientos posibles (delta fila, delta columna) son:

#	Delta Fila	Delta Columna
1	-2	-1
2	-2	+1
3	-1	-2
4	-1	+2
5	+1	-2
6	+1	+2
7	+2	-1
8	+2	+1

2.2 Condición de posición segura

Una casilla (fila, columna) es válida para colocar un caballo si y solo si se cumplen ambas condiciones:

- La casilla está vacía: `tablero[fila][columna] == 0`
- Ninguno de los 8 saltos del caballo desde esa casilla apunta a una casilla ya ocupada.

2.3 Restricción de solución válida

La solución debe contener exactamente 8 caballos colocados. En el Caso 2, la posición inicial del caballo dado por el usuario es fija e inamovible; los 7 caballos restantes se ubican respetando esa posición.

3. ¿Qué Representa una Solución Válida?

Una solución válida es una configuración del tablero 8×8 en la que:

1. Exactamente 8 casillas están marcadas con un caballo.
2. Para cada par de caballos (A, B), ninguno puede alcanzar al otro con un movimiento en L.
3. Si existe un caballo inicial (Caso 2), ese caballo está incluido en la configuración final.

Formalmente, si $S = \{(f1,c1), (f2,c2), \dots, (f8,c8)\}$ es el conjunto de posiciones, entonces para todo par (fi,ci) y (fj,cj) con $i \neq j$, la diferencia $(|fi - fj|, |ci - cj|)$ no puede ser ni (1,2) ni (2,1).

4. ¿Por Qué el Problema se Resuelve con Backtracking?

El backtracking es apropiado cuando el espacio de búsqueda es grande pero puede podarse anticipadamente al detectar conflictos. Este problema cumple exactamente esas condiciones:

4.1 Espacio de búsqueda enorme pero estructurado

Existen $C(64,8) \approx 4,426,165,368$ formas de elegir 8 casillas de 64. Con backtracking se descartan ramas completas en cuanto una casilla viola la restricción, reduciendo drásticamente las combinaciones exploradas.

4.2 Decisiones incrementales con capacidad de deshacerse

En cada paso se coloca un caballo en una casilla segura. Si la recursión posterior falla, se revierte la decisión (`tablero[fil][col] = 0`) y se prueba la siguiente casilla. Este patrón de "coloca / verifica / deshaz" es la esencia del backtracking.

4.3 Caso base bien definido

La condición `caballosColocados == 8` determina con claridad cuándo se ha encontrado una solución completa, terminando la recursión con éxito.

4.4 Estado compacto

Todo el estado necesario es el tablero 8×8 y el contador de caballos colocados. No se requiere historial adicional, lo que hace que la recursión sea eficiente en memoria.

5. Respuestas a las Preguntas Orientadoras

Pregunta	Respuesta
¿Cómo se determina si una posición es segura?	La función <code>esSegura(fila, col)</code> verifica que la casilla esté vacía y que ninguno de los 8 saltos del caballo desde ella apunte a una casilla ocupada.
¿Qué opciones tiene el algoritmo en cada paso?	Recorre las 64 casillas en orden (fila por fila, columna por columna). Por cada casilla segura, coloca el caballo y llama recursivamente. Si ninguna funciona, retorna false y se produce el backtrack.
¿Cuál es el caso base?	<code>caballosColocados == 8</code> : se han colocado todos los caballos sin conflictos. La función retorna true.
¿Cómo se revierte una decisión incorrecta?	Si la llamada recursiva retorna false, se ejecuta <code>tablero[fila][col] = 0</code> , borrando el caballo antes de intentar la siguiente casilla.

6. Análisis del Código Java Implementado

6.1 Estructura de la clase

La clase `ColocacionCaballos` encapsula el tablero como un arreglo bidimensional `int[8][8]` y define los 8 movimientos del caballo como una constante estática `MOVIMIENTOS_CABALLO`.

6.2 Método `ColocarCaballos(int caballosColocados)`

- Valida que el parámetro esté entre 0 y 8 (lanzando `IllegalArgumentException` si no).
- Retorna true inmediatamente si `caballosColocados == 8` (caso base).
- Itera sobre todas las casillas; para cada una segura, coloca el caballo y recursa.
- Si la recursión falla, deshace la colocación (backtrack).

6.3 Método `esSegura(int fila, int columna)`

- Valida que la posición esté dentro del tablero.
- Verifica que la casilla esté vacía.
- Comprueba los 8 saltos posibles: si alguno apunta a un caballo existente, retorna false.

6.4 Mejoras y correcciones aplicadas

El código incluye dos correcciones importantes:

4. Validación de entrada en `ColocarCaballos`: se verifica que `caballosColocados` esté en el rango `[0, 8]` antes de proceder.

5. Validación de límites en esSegura: se verifica que fila y columna estén dentro del tablero antes de acceder al arreglo.